

Universidade Federal de Ouro Preto - UFOP
Instituto de Ciências Exatas e Biológicas - ICEB
Departamento de Computação - DECOM
Ciência da Computação

Trabalho Prático 1

BCC203 - Estruturas de Dados II

Gabriel Canuto Câmara Souza (24.2.4087)
João Pedro Seabra Nogueira (25.1.4003)
Lara de Andrade Pereira (25.1.4012)
Luiz Felipe Bento Ferreira (24.1.4026)
Maria Eduarda de Carvalho Silva (25.1.4035)
Maurício de Oliveira Santos Rodrigues (25.1.4020)

Professor: Guilherme Tavares de Assis

Ouro Preto
26 de maio de 2026

Sumário

1	Introdução: Definição do Problema	1
2	Estrutura dos Dados e Metodologia de Testes	1
3	Implementação dos Métodos de Pesquisa Externa	2
3.1	Acesso Sequencial Indexado	2
3.1.1	Funções Implementadas	2
3.2	Árvore Binária de Pesquisa em Memória Externa	2
3.2.1	Funções Implementadas	3
3.3	Árvore B	3
3.3.1	Funções Implementadas	3
3.4	Árvore B*	4
3.4.1	Funções Implementadas	4
4	Metodologia de Testes e Coleta de Resultados	5
4.1	Automação das Buscas e Cálculo de Médias	5
5	Análise de Desempenho Individual	6
5.1	Avaliação: Acesso Sequencial Indexado	6
5.2	Avaliação: Árvore Binária de Pesquisa	6
5.3	Avaliação: Árvore B	7
5.4	Avaliação: Árvore B*	7
6	Comparação entre os Métodos	8
6.1	Pesquisa de 100 registros	8
6.2	Pesquisa de 1.000 registros	8
6.3	Pesquisa de 10.000 registros	8
6.4	Pesquisa de 100.000 registros	9
6.5	Pesquisa de 1.000.000 registros	9
7	Dificuldades de Implementação	10
8	Conclusões Finais	10

Lista de Tabelas

1	Desempenho do método de Acesso Sequencial Indexado	6
2	Desempenho do método de Árvore Binária de Pesquisa	6
3	Desempenho do método de Árvore B	7
4	Desempenho do método de Árvore B*	7
5	Comparação dos métodos na pesquisa de 100 registros	8
6	Comparação dos métodos na pesquisa de 1.000 registros	8
7	Comparação dos métodos na pesquisa de 10.000 registros	8
8	Comparação dos métodos na pesquisa de 100.000 registros	9
9	Comparação dos métodos na pesquisa de 1.000.000 registros	9

1 Introdução: Definição do Problema

Este trabalho prático tem como objetivo implementar na linguagem C e analisar o desempenho de quatro métodos de pesquisa em memória externa: **acesso sequencial indexado**, **árvore binária de pesquisa adequada à memória externa**, **árvore B** e **árvore B***.

A análise experimental avaliará a eficiência de cada algoritmo na manipulação de arquivos binários contendo grandes volumes de dados. O desempenho será medido considerando três pontos fundamentais: o número de transferências (leituras) da memória externa para a interna, o número de comparações entre as chaves de pesquisa e o tempo de execução.

Os resultados obtidos permitirão comparar o comportamento de cada método na prática, evidenciando suas vantagens, limitações e viabilidade em diferentes cenários de quantidade de dados no disco.

2 Estrutura dos Dados e Metodologia de Testes

Os testes serão feitos utilizando arquivos binários com volumes de dados: 100, 1.000, 10.000, 100.000 e 1.000.000 de registros. Cada registro do arquivo terá um tamanho fixo e será composto por: uma **chave** de pesquisa (valor inteiro), **dado1** (valor inteiro longo), **dado2** (cadeia de 1000 caracteres) e **dado3** (cadeia de 5000 caracteres).

Esses arquivos vão ser organizados de três formas diferentes: ordenados de forma ascendente, ordenados de forma descendente e desordenados aleatoriamente. Vale ressaltar que, de acordo com as restrições de arquitetura de cada método de pesquisa implementado, algumas situações de ordenação do arquivo não se aplicam e serão desconsiderados nos testes.

Para cada cenário de teste (combinação de método, quantidade de registros e situação do arquivo), ocorrerá a pesquisa automática de 10 chaves distintas, previamente selecionadas e garantidamente presentes no arquivo. Esta abordagem visa obter uma média consistente e representativa de cada um dos quesitos avaliados (transferências, comparações e tempo).

3 Implementação dos Métodos de Pesquisa Externa

3.1 Acesso Sequencial Indexado

O método de Acesso Sequencial Indexado (ASI) consiste em organizar os registros em páginas no arquivo binário e manter uma tabela de índices em memória principal. Cada entrada da tabela armazena a primeira chave de uma página e o número da página correspondente. A busca é realizada em duas etapas: primeiro, uma busca binária na tabela de índices para localizar a página candidata; em seguida, uma busca binária dentro da página lida para encontrar o registro desejado.

3.1.1 Funções Implementadas

geradorSequencial(): é responsável pela geração do arquivo binário utilizado no método de Acesso Sequencial Indexado. Como o método requer arquivos ordenados pela chave de pesquisa, a função gera registros em ordem crescente e os escreve sequencialmente no arquivo. Além disso, os demais campos dos registros são preenchidos com dados aleatórios.

criaTabelaIndice(): é responsável pela construção da tabela de índices em memória principal. Para cada página do arquivo, a primeira chave é armazenada juntamente com sua posição, permitindo que a pesquisa localize rapidamente a página desejada através de busca binária.

pesquisaASI(): é responsável pela execução da pesquisa utilizando o método de Acesso Sequencial Indexado. Inicialmente, é realizada uma busca binária na tabela de índices para localizar a página onde a chave desejada pode estar armazenada. Em seguida, a página correspondente é carregada da memória externa para a memória principal e uma nova busca binária é realizada sobre os registros presentes na página. Durante a execução, também são contabilizadas as quantidades de comparações e transferências realizadas pelo método.

buscaSequencial(): o arquivo de registros é gerado e a tabela de índices é criada. Em seguida, a função realiza a pesquisa da chave desejada utilizando o método implementado em **pesquisaASI()**, contabilizando comparações, transferências e tempo de execução.

Parâmetros de Configuração:

- ITENSPAGINA = 100 registros por página
- MAXTABELA = 10050 entradas na tabela de índices

O tamanho máximo da tabela de índices foi definido como 10050 posições para evitar problemas de estouro de memória durante os testes com grandes volumes de dados. Considerando o pior caso especificado no trabalho, com 1.000.000 de registros e páginas contendo 100 registros cada, o número total de páginas geradas é dado por:

$$\frac{1.000.000}{100} = 10.000 \text{ páginas}$$

Como cada página possui uma entrada correspondente na tabela de índices, seriam necessárias exatamente 10.000 posições. Por isso foi escolhido um tamanho um pouco maior, adicionando uma margem de segurança.

3.2 Árvore Binária de Pesquisa em Memória Externa

A Árvore Binária de Pesquisa adaptada para memória externa consiste em armazenar cada nó da árvore diretamente como um registro no arquivo binário. Cada nó contém, além da sua chave de pesquisa e dos campos de dados (**dado1**, **dado2** e **dado3**), dois apontadores representados por *offsets* (posições relativas no disco) que indicam a localização do filho à esquerda e do filho à direita.

Diferente das estruturas multicaminho avaliadas neste trabalho (como a Árvore B e B*), a Árvore Binária de Pesquisa não possui mecanismos inerentes de balanceamento. Como consequência, a topologia da árvore depende diretamente da ordem em que os registros são inseridos. Em casos de arquivos previamente ordenados (ascendente ou descendente), a estrutura sofre degeneração, comportando-se na prática como uma lista

encadeada, o que eleva drasticamente o número de transferências da memória externa para a principal e inviabiliza sua utilização para arquivos muito grandes.

3.2.1 Funções Implementadas

geradorArvoreBinaria(): É responsável por inicializar o arquivo binário e coordenar o fluxo de carga dos dados. A função gera as chaves e os campos complementares e, para cada novo registro, aciona a rotina de inserção, construindo a árvore nó a nó diretamente na memória secundária.

insereArvoreBinaria() e insereNo(): Implementam a lógica de navegação e gravação dos nós. Para inserir um novo elemento, o algoritmo lê os registros do arquivo navegando pelos *offsets* de acordo com o valor da chave (esquerda para menores, direita para maiores). Ao encontrar um apontador nulo (indicando espaço disponível), o novo registro é gravado fisicamente no final do arquivo utilizando **fwrite()**, e o apontador do nó pai correspondente é atualizado no disco com o novo *offset*.

pesquisaArvoreBinaria() e pesquisaNo(): Executam o processo de busca partindo do nó raiz (geralmente posicionado no *offset* zero do arquivo). A função carrega um nó por vez para a memória principal — contabilizando uma transferência — e compara a chave lida com a chave buscada. Dependendo do resultado da comparação, a busca segue recursivamente ou iterativamente pelo *offset* da esquerda ou da direita, acumulando as métricas de transferências e comparações até encontrar o registro ou atingir um apontador vazio.

printArvoreBinaria(): Realiza um percurso em-ordem (*in-order*) na estrutura armazenada no disco. Através de leituras sucessivas seguindo a hierarquia dos apontadores da esquerda para a direita, a função imprime no console as chaves e os subcampos associados, permitindo verificar a integridade estrutural e a correta ordenação da árvore gerada.

3.3 Árvore B

A Árvore B é uma estrutura balanceada projetada especificamente para gerenciar grandes volumes de dados armazenados em meios de memória secundária. Em vez de nós individuais contendo apenas um elemento, cada nó da Árvore B é mapeado como uma página de tamanho fixo que comporta um vetor de até $2M$ registros ordenados de forma crescente e um vetor de $2M + 1$ ponteiros de offsets direcionados para páginas filhas. O balanceamento perfeito assegura que todas as páginas folha permaneçam rigorosamente na mesma profundidade, garantindo altura máxima limitada por $O(\log_M N)$.

Durante a fase de inserção, se uma página atinge o limite máximo de armazenamento ($2M$) e recebe um novo item, o algoritmo engatilha uma divisão de página (*split*). A chave que ocupa a posição mediana do bloco é promovida para o nível superior (página pai) e as chaves restantes são divididas igualmente em duas novas páginas de tamanho M . Essa propriedade mitiga de forma drástica os acessos a disco durante as operações de pesquisa.

3.3.1 Funções Implementadas

geradorArvoreB(): Configura o vetor estrutural de chaves e realiza a carga do primeiro registro criando a página raiz inicial no arquivo. Posteriormente, processa os demais itens acionando o método de inserção de páginas.

insereNaArvore() e ins(): Funções que atuam de forma recursiva para gerenciar o fluxo de inserção e quebra de páginas. Elas navegam de forma descendente para encontrar a página folha adequada. No retorno da recursão, caso a flag de crescimento seja ativada, indica que a subpágina sofreu um *split*. Se a página pai atual possuir espaço disponível ($n < 2M$), a chave promovida é inserida ordenadamente através da rotina auxiliar **inserePagina()**; caso contrário, um novo *split* é disparado em cascata, alocando uma nova página via **fwrite()** no final do arquivo. Se o estouro atingir a raiz principal, o tamanho do arquivo cresce em altura e a nova raiz passa a ocupar de forma fixa a posição inicial (offset 0).

pesquisaChave() e pesquisaRec(): Realizam a busca estruturada partindo da página raiz. Ao ler uma página para a memória principal (contabilizando uma transferência), o algoritmo executa uma busca linear ou binária entre as chaves contidas. Se houver correspondência exata, o registro é retornado com sucesso.

Caso contrário, a busca determina entre quais chaves a desejada se posiciona, identifica o offset da subárvore filha e realiza uma chamada recursiva para a nova página.

printArvore() e **printArvoreRec()**: Realizam o percurso recursivo em-ordem (*in-order*) por todas as páginas ativas no arquivo binário, listando no terminal as chaves e os subcampos associados de forma hierárquica.

3.4 Árvore B*

A Árvore B* é uma estrutura de dados balanceada utilizada em métodos de pesquisa externa, sendo uma variação da Árvore B tradicional. Seu principal objetivo é otimizar o uso do espaço em disco e reduzir a quantidade de acessos à memória secundária durante operações de busca, inserção e remoção.

Diferentemente da Árvore B convencional, a Árvore B* realiza redistribuições entre páginas irmãs antes de efetuar divisões de nós, garantindo uma taxa mínima de ocupação maior. Dessa forma, a estrutura mantém as páginas mais preenchidas, reduzindo a altura da árvore e melhorando o desempenho das operações.

No método de pesquisa utilizando Árvore B*, as chaves são organizadas de maneira hierárquica e ordenada, permitindo localizar registros através de sucessivas decisões de navegação entre os nós da árvore até alcançar a página que contém a chave desejada.

A utilização dessa estrutura é especialmente eficiente em aplicações que manipulam grandes volumes de dados armazenados em memória externa, devido à redução do número de transferências entre disco e memória principal.

3.4.1 Funções Implementadas

geradorArvoreEstrela(): Aloca e popula o vetor de controle de chaves baseando-se na ordenação exigida e despacha em laço contínuo os registros para a árvore balanceada B*.

insereArvoreB_estrela() e **insRecEstrela()**: Implementam a lógica complexa de inserção descendente e particionamento de nós na Árvore B*. O algoritmo ramifica suas decisões conforme o tipo de página coletada pelo campo `tipoDePagina`. Nas páginas internas, navega-se de forma otimizada comparando apenas vetores de inteiros guia (`ri`); nas externas, os dados são inseridos em ordem e, em caso de *overflow*, cria-se um novo nó folha no final do arquivo, promovendo a menor chave do novo nó criado para o nó pai indexador.

pesquisarChaveB_estrela() e **pesquisaRecEstrela()**: Executam a varredura lógica do método B*. O fluxo percorre as páginas de índice interno sem precisar carregar registros de tamanho fixo massivo para a memória primária. Ao interceptar a folha externa que contém o intervalo da chave de interesse, uma única leitura de registro completo é disparada, otimizando as estatísticas globais de transferências.

printArvoreB_estrela() e **printArvoreEstrelaRec()**: Percorrem de forma estruturada a árvore, diferenciando nós internos e externos para exibir no console a listagem completa dos dados textuais contidos.

4 Metodologia de Testes e Coleta de Resultados

Para a realização da análise de desempenho dos métodos, os testes foram feitos de acordo com as diretrizes da segunda fase do trabalho prático. Foram gerados arquivos contendo diferentes volumes de registros ($N \in \{100, 1.000, 10.000, 100.000, 1.000.000\}$), avaliados sob as situações de ordenação ascendente, descendente e aleatória.

A avaliação do tempo gasto para a criação do arquivo de dados original não foi contabilizada no tempo de execução dos métodos.

4.1 Automação das Buscas e Cálculo de Médias

Para cada teste (mesmo método, quantidade de registros e situação de ordem), o sistema realizou a busca automática de **10 chaves de pesquisa distintas**, escolhidas aleatoriamente, mas com garantia de existência prévia no arquivo de dados.

Para cada uma das 10 buscas efetuadas, foram coletados os seguintes quesitos:

- Número de transferências (leitura) de itens da memória externa para a interna.
- Número de comparações entre as chaves de pesquisa.
- Tempo de execução da pesquisa (tempo de término subtraído do tempo de início).

Os resultados finais apresentados nas seções a seguir representam a **média aritmética** dos valores obtidos nessas 10 execuções para cada cenário. Essa abordagem permitiu extrair uma métrica de desempenho mais estável e representativa para o comportamento de cada estrutura de dados em memória externa.

5 Análise de Desempenho Individual

5.1 Avaliação: Acesso Sequencial Indexado

A Tabela 1 apresenta os resultados obtidos para o método de Acesso Sequencial Indexado.

IMPORTANTE: O método de Acesso Sequencial Indexado pressupõe arquivos ordenados pela chave de pesquisa. Dessa forma, apenas a situação de ordenação crescente foi considerada válida para execução do método.

Tabela 1: Desempenho do método de Acesso Sequencial Indexado

		Acesso Sequencial Indexado				
Ordem	Operação	100 itens	1.000 itens	10.000 itens	100.000 itens	1.000.000 itens
Ascendente	Pesquisa (segundos)	0.000096	0.000113	0.000126	0.000138	0.000759
	Transferências	1	1	1	1	1
	Comparações	6	9	12	16	18
Descendente	Pesquisa (segundos)	-	-	-	-	-
	Transferências	-	-	-	-	-
	Comparações	-	-	-	-	-
Aleatória	Pesquisa (segundos)	-	-	-	-	-
	Transferências	-	-	-	-	-
	Comparações	-	-	-	-	-

5.2 Avaliação: Árvore Binária de Pesquisa

IMPORTANTE: O método de Árvore Binária de Pesquisa não considera entradas grandes (maiores que 10.000 itens) ordenadas previamente, pois a árvore gerada é, basicamente, uma lista encadeada, o que torna inviável a realização de testes. Dessa forma, apenas a situação de ordenação aleatória foi considerada válida para execução do método.

Tabela 2: Desempenho do método de Árvore Binária de Pesquisa

		Árvore Binária de Pesquisa				
Ordem	Operação	100 itens	1.000 itens	10.000 itens	100.000 itens	1.000.000 itens
Ascendente	Pesquisa (segundos)	0.000068	0.000030	-	-	-
	Transferências	40	536	-	-	-
	Comparações	79	1072	-	-	-
Descendente	Pesquisa (segundos)	0.000098	0.001070	-	-	-
	Transferências	53	613	-	-	-
	Comparações	106	1226	-	-	-
Aleatória	Pesquisa (segundos)	0.000013	0.000021	0.00028	0.000050	0.000132
	Transferências	6	12	15	20	25
	Comparações	12	23	29	40	50

5.3 Avaliação: Árvore B

A Tabela 2 apresenta os resultados obtidos para o método de Árvore B.

Tabela 3: Desempenho do método de Árvore B

		Árvore B				
Ordem	Operação	100 itens	1.000 itens	10.000 itens	100.000 itens	1.000.000 itens
Ascendente	Pesquisa (segundos)	0.000019	0.000024	0.000042	0.00051	0.000217
	Transferências	3	4	6	7	9
	Comparações	5	12	13	17	23
Descendente	Pesquisa (segundos)	0.000018	0.000022	0.000049	0.000055	0.000221
	Transferências	3	4	6	7	9
	Comparações	6	12	13	17	22
Aleatória	Pesquisa (segundos)	0.000017	0.000019	0.000031	0.000042	0.000087
	Transferências	3	4	5	7	8
	Comparações	7	10	14	20	24

5.4 Avaliação: Árvore B*

A Tabela 4 apresenta os resultados obtidos para o método de Árvore B*.

Tabela 4: Desempenho do método de Árvore B*

		Árvore B*				
Ordem	Operação	100 itens	1.000 itens	10.000 itens	100.000 itens	1.000.000 itens
Ascendente	Pesquisa (segundos)	0.000020	0.000027	0.000041	0.000066	0.000177
	Transferências	4	5	7	8	10
	Comparações	9	16	17	21	22
Descendente	Pesquisa (segundos)	0.000018	0.000028	0.000046	0.000059	0.000253
	Transferências	4	5	7	8	10
	Comparações	8	13	15	21	23
Aleatória	Pesquisa (segundos)	0.000021	0.000024	0.000031	0.000049	0.000113
	Transferências	4	5	6	8	9
	Comparações	8	14	18	21	29

6 Comparação entre os Métodos

6.1 Pesquisa de 100 registros

Tabela 5: Comparação dos métodos na pesquisa de 100 registros

		100 Registros			
Ordem	Operação	Indexação Sequencial	Árvore Binária	Árvore B	Árvore B*
Ascendente	Pesquisa (segundos)	0.000096	0.000068	0.000019	0.000020
	Transferências	1	40	3	4
	Comparações	6	79	5	9
Descendente	Pesquisa (segundos)	-	0.000098	0.000018	0.000018
	Transferências	-	53	3	4
	Comparações	-	106	6	8
Aleatória	Pesquisa (segundos)	-	0.000013	0.000017	0.000021
	Transferências	-	6	3	4
	Comparações	-	12	7	8

6.2 Pesquisa de 1.000 registros

Tabela 6: Comparação dos métodos na pesquisa de 1.000 registros

		1.000 Registros			
Ordem	Operação	Indexação Sequencial	Árvore Binária	Árvore B	Árvore B*
Ascendente	Pesquisa (segundos)	0.000113	0.000030	0.000024	0.000027
	Transferências	1	536	4	5
	Comparações	9	1072	12	16
Descendente	Pesquisa (segundos)	-	0.001070	0.000022	0.000028
	Transferências	-	613	4	5
	Comparações	-	1226	12	13
Aleatória	Pesquisa (segundos)	-	0.000021	0.000019	0.000024
	Transferências	-	12	4	5
	Comparações	-	23	10	14

6.3 Pesquisa de 10.000 registros

Tabela 7: Comparação dos métodos na pesquisa de 10.000 registros

		10.000 Registros			
Ordem	Operação	Indexação Sequencial	Árvore Binária	Árvore B	Árvore B*
Ascendente	Pesquisa (segundos)	0.000126	-	0.000042	0.000041
	Transferências	1	-	6	7
	Comparações	12	-	13	17
Descendente	Pesquisa (segundos)	-	-	0.000049	0.000046
	Transferências	-	-	6	7
	Comparações	-	-	13	15
Aleatória	Pesquisa (segundos)	-	0.00028	0.000031	0.000031
	Transferências	-	15	5	6
	Comparações	-	29	14	18

6.4 Pesquisa de 100.000 registros

Tabela 8: Comparação dos métodos na pesquisa de 100.000 registros

		100.000 Registros			
Ordem	Operação	Indexação Sequencial	Árvore Binária	Árvore B	Árvore B*
Ascendente	Pesquisa (segundos)	0.000138	-	0.00051	0.000066
	Transferências	1	-	7	8
	Comparações	16	-	17	21
Descendente	Pesquisa	-	-	0.000055	0.000059
	Transferências	-	-	7	8
	Comparações	-	-	17	21
Aleatória	Pesquisa (segundos)	-	0.000050	0.000042	0.000049
	Transferências	-	20	7	8
	Comparações	-	40	20	21

6.5 Pesquisa de 1.000.000 registros

Tabela 9: Comparação dos métodos na pesquisa de 1.000.000 registros

		1.000.000 Registros			
Ordem	Operação	Indexação Sequencial	Árvore Binária	Árvore B	Árvore B*
Ascendente	Pesquisa (segundos)	0.000759	-	0.000217	0.000177
	Transferências	1	-	9	10
	Comparações	18	-	23	22
Descendente	Pesquisa (segundos)	-	-	0.000221	0.000253
	Transferências	-	-	9	10
	Comparações	-	-	22	23
Aleatória	Pesquisa (segundos)	-	0.000132	0.000087	0.000113
	Transferências	-	25	8	9
	Comparações	-	50	24	29

7 Dificuldades de Implementação

Durante o desenvolvimento do trabalho, algumas dificuldades foram encontradas na implementação e execução dos métodos de pesquisa externa. Uma das principais limitações esteve relacionada ao grande volume de dados utilizado nos testes, especialmente nos testes com arquivos contendo até 1.000.000 de registros. Devido ao tamanho elevado dos arquivos gerados, houve aumento significativo no consumo de memória e no tempo de processamento. Além disso, alguns computadores utilizados pelos integrantes do grupo apresentaram limitações na hora de rodar os testes.

Outra dificuldade relevante foi a implementação da Árvore B*, considerada um dos métodos mais complexos do trabalho. A manipulação das páginas, redistribuições entre nós irmãos e divisões de páginas exigiram maior cuidado durante a implementação, tornando o desenvolvimento mais trabalhoso em comparação aos demais métodos.

8 Conclusões Finais

Colocar a mão na massa e analisar os resultados deste trabalho nos permitiu entender, na prática, onde cada método de pesquisa em memória externa brilha e onde deixa a desejar.

O **Acesso Sequencial Indexado (ASI)** surpreendeu pela velocidade. Ele foi o mais eficiente no acesso ao disco, precisando de apenas uma única transferência da memória externa para a interna, não importando o tamanho do arquivo. Esse desempenho excelente acontece graças à combinação da busca binária na tabela de índices com a busca na página já carregada na RAM. O grande problema é que ele é muito engessado: como exige que o arquivo já esteja ordenado previamente, acaba não sendo uma opção viável para sistemas dinâmicos, onde dados são inseridos e removidos o tempo todo.

Já a **Árvore Binária de Pesquisa** confirmou o que a teoria já avisava: ela simplesmente não foi feita para lidar com memória secundária. Como não tem um mecanismo de balanceamento e utiliza nós individuais, o custo de transferências e comparações vai às alturas. Para piorar, quando os arquivos estão ordenados, a estrutura se degenera e vira basicamente uma lista encadeada. Isso até nos impediu de rodar testes com volumes maiores que 10.000 registros, deixando claro que não dá para contar com ela no gerenciamento de discos.

Em contrapartida, a **Árvore B** e a **Árvore B*** entregaram tudo o que prometeram. Elas se mostraram soluções ideais e muito robustas para lidar com grandes bancos de dados, encarando sem problemas arquivos em todas as ordens (crescente, decrescente e aleatória) e escalando perfeitamente até a marca de 1.000.000 de registros. O segredo do sucesso delas é o agrupamento de registros em páginas, o que mantém a estrutura da árvore bem “baixa”. Para se ter uma ideia, encontrar uma chave no meio de um milhão de registros custou, no máximo, 10 leituras no disco.

Para fechar, vale reconhecer que a **Árvore B*** dá bem mais trabalho para ser implementada. Lidar com os algoritmos de redistribuição de nós e o atraso nas quebras de página (*splits*) exige bastante cuidado. Mesmo assim, o esforço compensa por oferecer um melhor aproveitamento do espaço de armazenamento a longo prazo. No fim das contas, tanto a Árvore B quanto a B* tiveram um desempenho excepcional em velocidade e número de transferências, provando que apostar em estruturas multicaminho balanceadas é o caminho certo para manipular grandes volumes de dados com eficiência.